/ MMB8052 - Practical 2

MMB8052 Practical 02 - Advanced Commmand Line Linux

Introduction

In the first practical we introduced the basics of the Linux command line. This second practical builds on these fundamentals to introduce more advanced tools and processes using the command line, with a particular focus on tools which process and manipulate text-based data.

A large amount of bioinformatics data is based on standard text formats - we've already seen UniProt database entries which are made up of structured text, and the FASTA sequence format, in which individual sequences are delineated by header lines beginning with ">". Other common formats include the FASTQ format for high-throughput sequencing data, the SAM (Sequence Alignment Map) format for storing genomic alignments and the GFF (General Feature Format) which is used for recording the position of genomic features.

Linux tools for processing text have been around for the best part of 50 years, and as such are robust and perform well, even with the often very large files common in bioinformatics. This practical will explore a number of these tools and how they can be used to work with the type of files listed above.

Tools for Text

Formats

FASTA

The FASTA file format is a 40-year old standard for representing either nucleotide or amino acid sequences. The first line of a FASTA entry begins with the greater-than symbol (`>`), and is referred to as the description line. It should contain a unique identifier for the sequence. The lines following the description line contain the sequence data, which can be spread across multiple lines and are expected to be represented in the standard IUPAC single-letter codes. A single file can contain many sequence entries, each beginning with its own description line.

```
>sequence_1
CGATAGCAGATGCGATGGATAGAGATTAGATAGAGGCTCGCGCTGACGCCGC
CGATGCGCTATCGCGCTAGCTCG
>sequence_2
ATGAGCTGAGCGTAGTCGCGCATCAGCAGACTACAGCAGCAGACTCAGAACA
GCATACGCAGACTACGCAGACGACTCAGCAGACGACTACGA
```

FASTQ

The FASTQ format was developed by the Wellcome Trust Sanger Institute to store nucleotide sequence data along with associated quality scores, which provide information about the confidence of sequencing accuracy. FASTQ has become the *de facto* standard for storing the output of high-throughput sequencing instruments. An entry in a FASTQ file consists of 4 lines:

1. Begins with an `@` character, and is followed by a sequence identifier (like the FASTA description line)

2. The sequence data (A/C/G/T/N)

3. Delimits the sequence data and the quality data - in modern files usually contains only a `+` character

4. The quality data for the bases in line 2 - must contain the same number of characters as line 2

The quality scores are encoded as an ASCII value - that is a single character that can be translated into an integer numerical value, generally in the range 0 - 41. This quality score, or Phred score is related to the probability of the corresponding base call being an error (the higher the Phred score, the lower this probability).

```
@NB501177:26:HF3Y2BGX2:1:11101:23339:1070 1:N:0:CGGC
GTGGCACCATGGTTGTCAGACGTCAGTGCANACANNNCTAGATGTGGGCTAC
+
AAAAAEEEEEEEEEEEEAE6AEEEEEEEEE#EEE###EEEEEEEEEEEEEE
@NB501177:26:HF3Y2BGX2:1:11101:14507:1071 1:N:0:CGGC
GCCACAGTCAGCCAGCCAAACGTATGTTAGNATTNNNCAGGTGATCACAAAA
+
AAAAAEEEEEEEEEEEEEEEEEEEEEEEE#EEE###EEEEEEEEEEEEEE
```

The ASCII quality characters used for Illumina sequencing, from left-to-right in increasing order of quality:

```
!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJ
```

Most modern Illumina sequencers use quality "bins", a small set of discrete values which means that FASTQ files can be compressed more easily. For example, bases sequenced on the NovaSeq 6000 can take one of four possible quality values 2 (`#`), 12 (`-`), 23 (`8`) and 37 (`F`). The sequences in the example above come from a NextSeq 500, which uses six bins (`#`, `/`, `6`, `<`, `A`, `E`).

SAM

SAM stands for *Sequence Alignment/Map* format. It is a text-based table used to record the alignment of sequence reads to a reference sequence. Each individual alignment is recorded in a single line with 11 mandatory fields separated by tab characters.

The header

The header is an optional section of the SAM file that can describe things about the experiment as a whole. Each header line begins with the "@" character followed by a two-letter record type code. Each line is then tab-delimited.

```
@HD   VN:1.0     SO:unsorted
@SQ   SN:chr1    LN:197195432
@PG   ID:bowtie2 PN:bowtie2 VN:2.3.5.1
```

The `@HD` line is information about the SAM file itself (version, whether or not it is sorted). `@SQ` is information about the reference sequence (there would be one line per chromosome, if we were aligning to a whole multi-chromosome genome). `@PG` is about the program that generated the SAM file.

Mandatory alignment fields

Column	Description	Notes
1	Query Template Name	Read name in the FASTQ file being mapped

Column	Description	Notes
2	Bitwise Flag	There's a good description of bitwise flags here https://davetang.org/muse/2014/03/06/understa bam-flags/, and a decoder for the values in this column here: http://broadinstitute.github.io/picard/explain-flag
3	Reference Sequence Name	Sequence in the reference genome mapped to (usually a chromosome)
4	1-based left most mapping position	Beginning position of the mapping
5	Mapping Quality	$-10\log_{10}$ P(mapping is wrong). A value of 255 indicates that the mapping quality is not availab
6	CIGAR string	CIGAR = Concise Idiosyncratic Gapped Alignm Report

Column	Description	Notes
		More information
7	Reference name of the mate/next segment	Relevant to paired-end data
8	Position of the mate/next segment	Relevant to paired-end data
9	Observed template length	Relevant to paired-end data
10	Segment Sequence	The sequence of the read (not the reference)
11	ASCII of Phred-scaled base	Same as the quality in Sanger-encoded FASTC

Column	Description	Notes
	base	
	Quality+33	

Additional alignment fields

All additional fields in an alignment line consist of 3 values, separated by colons, in the form TAG:TYPE:VALUE. There are a number of predefined tag types that can provide additional information about the alignment.

SAM alignment of the reads shown in the FASTQ example, above (generated by the alignment software HISAT2):

```
NB501177:26:HF3Y2BGX2:1:11101:23339:1070        0
NB501177:26:HF3Y2BGX2:1:11101:14507:1071        16
```

GFF/GTF

The General Feature Format (GFF) is a standard table-based format for encoding genomic features. Each feature is encoded on one line, and each line consists of 9 columns of data. There have been 3 major versions of GFF, and the GTF (Gene Transfer Format) is very similar to GFF version 2. GFF3 is the most recent, and the preferred version of this format.

Column	Description	Notes
1	seqid	Identifier of the genomic sequence the current feature is found on.

Column	Description	Notes
2	source	Software or database feature comes from.
3	type	The type of feature, taken from the "lite" sequence ontology (SOFA).
4	start	1-indexed start location of the feature.
5	end	1-indexed end location of the feature.
6	score	The score is usually derived by gene prediction software, and is likely an E-value or P-value. Can be left blank (".).
7	strand	+ for positive strand, - for negative strand (relative to genomic sequence in 1).
8	phase	For features of type "CDS", the phase indicates where the feature begins with reference to the reading frame. The phase is one of the integers 0, 1, or 2, indicating the number of bases that should be removed from the beginning of this feature to reach the first base of the next codon.
9	attributes	A list of feature attributes in the format tag=value. Multiple tag=value pairs are separated by semicolons.

The flexible format of the 9th column makes this format problematic, and a GFF file needs to be examined carefully to understand what fields are found in column 9. A number of tags with pre-defined meaning are available see the format specification for more information. The GFF format is often used for defining gene structures.

```
##gff-version 3
ctg123 . mRNA            1300  9000  .  +  .  ID=mrn
ctg123 . exon            1300  1500  .  +  .  ID=exo
ctg123 . exon            1050  1500  .  +  .  ID=exo
ctg123 . exon            3000  3902  .  +  .  ID=exo
ctg123 . exon            5000  5500  .  +  .  ID=exo
ctg123 . exon            7000  9000  .  +  .  ID=exo
```

wc

The `wc` command (word count) is a utility that given a file will list the number of lines, words and bytes in that file. The `-l` option restricts the output to just the number of lines in the file - this is particularly useful for file in which every line is a separate record, such as the case with tabular data.

Similarly, the `-w` option returns only the number of words in a file, and the `-c` option returns only the number of bytes (characters).

Exercise 2.1

Estimated time: 1 min

Use `wc` to find the number of words, lines and bytes in the file of protein sequences you downloaded in Exercise 1.8.

grep

Searching for specific text in large files can be a laborious process - `grep` is a useful and efficient utility which simplifies this task. `grep phrase file` is a command which will search for lines which contain `phrase` in a text file called `file`.

If our search phrase contains special characters which are usually interpreted by the bash shell (e.g. `>`, `<`, `|` or `*`), then it is necessary to enclose the phrase in single quotes - `grep 'phrase' file` - to prevent bash from interpreting these characters. For example, we would count the number of sequence entries in a FASTA file by counting the lines which contain the `>` character (which is used to start sequence header lines according to the FASTA specification). Since `>` is interpreted by bash as the redirection of `STDOUT`, the following command would actually overwrite the *input* file, by redirecting the (non-existant) output of the `grep` command into the file:

```
$ grep > human.fa
```

To prevent this, we should ensure the search phrase is enclosed in single quotes:

```
$ grep '>' human.fa
```

This command will return all of the lines which contain a `>` character. We could then, for example, use `wc` to count them:

```
$ grep '>' human.fa | wc -l
```

This command makes use of the "pipe" character (`|`) we first encountered in practical 1 to use the output from `grep` as the input for `wc`.

Some useful options for `grep` include:

- `-c` to count the number of lines containing the search phrase (this would give the same result as the piped command, above)
- `-i` to make the search case-insensitive
- `-o` to select only matching strings (rather than the whole line)
- `-w` to select only lines containing matches that form whole words
- `-v` to select non-matching lines
- `--color` to highlight matches in the selected line (on by default in most Linux distributions)

Remember that `man grep` will give a full list of the available options and their descriptions.

We can also use the special characters `^` and `$` to match phrases at the start and end of a line, for example:

```
$ grep '^>' human.fa
```

Will ensure that we only match `>` characters found at the *beginning* of a line (this character is a *qualifier* for the search phrase - it is not included as part of the phrase to be matched).

Exercise 2.2

Estimated time: 3-4 min

- If you don't still have it saved from practical 1, use `wget` to download the Retinoblastoma UniProt entry (https://rest.uniprot.org/uniprotkb/P06400.txt)
- The lines of a UniProt file begin with 2 characters which denote the *type* of line (documented in the user manual: https://web.expasy.org/docs/userman.html). Use `grep`, and this

information to retrieve a list of publication identifiers relevant to the Rb protein (encoded in the `RX` lines)

- Narrow this list down to only those references which have a Digital Object Identifier (DOI). (Hint: you'll need to use a pipe (`|`) here)

Consider the following:

- Does your first search find anything other than publication identifier lines?
    - How could you make the search more specific to prevent this, if it is a problem?
- How many publications with a DOI are associated with the Rb protein?
- How about those with a PubMed ID?

Regular Expressions

Searching for exact text matches is rather limited in scope - the true power of tools like `grep` becomes apparent when we can define search *patterns* which can match variable text results. Regular expressions allow us to define these types of pattern.

`grep` actually provides the user with a number of ways of defining search patterns, the most powerful of these are Perl regular expressions (Perl is a programming language which is extremely good at text processing). These regular expressions can be "switched on" in `grep` by using the `-P` option.

A Perl regular expression is defined as a string of characters where each entity is either a regular character, which takes its literal meaning, or a *metacharacter*, which has a special meaning. Some metacharacter examples:

- `.` - match *any* character
- `\w` - match any "word" character (alphanumerics and underscores)

- `\s` - match any "whitespace" character
- `\d` - match digit character (i.e. the numbers 0-9)
- `\D` - match non-digit character (i.e. anything *other than* the numbers 0-9)
- `\t` - match the tab character
- `\n` - match the newline character (i.e. what proceeds this character must be at the end of the line)
- `*` - match the preceding character (or metacharacter) zero or more times
- `+` - match the preceding character (or metacharacter) one or more times
- `{n}` - match the preceding character (or metacharacter) exactly `n` times (where `n` is replaced by a positive whole number)
- `{n,}` - match the preceding character at least `n` times
- `{m,n}` - match the preceding character (or metacharacter) between `m` and `n` times (inclusive)

If we want to match the actual characters (`.` , `+` , `*` etc) rather than imply their programmatic meaning as metacharacters, we will need to "escape" the character with a backslash (`\`). So if our pattern contains `+` , we need to express this so: `\+` .

By way of example, we will search the UniProt entry of another protein (ATM, Q13315) for the Ensembl Gene ID of the protein's parent gene. Human Ensembl Gene IDs start with ENSG, so we can use that to make a regular search of the UniProt entry:

```
$ wget 'https://rest.uniprot.org/uniprotkb/Q13315.tx
$ grep 'ENSG' Q13315.txt
DR   Ensembl; ENST00000278616.9; ENSP00000278616.4;
DR   Ensembl; ENST00000452508.6; ENSP00000388058.2;
DR   Ensembl; ENST00000675843.1; ENSP00000501606.1;
DR   HPA; ENSG00000149311; Low tissue specificity.
```

```
DR   OpenTargets; ENSG00000149311; -.
DR   VEuPathDB; HostDB:ENSG00000149311; -.
DR   GeneTree; ENSGT00670000098061; -.
DR   Bgee; ENSG00000149311; Expressed in corpus call
     RFLCKAVENY INCLLSGEEH DMWVFRLCSL WLENSGVSEV NGM
```

This search retrieves the expected database reference (DR) lines for Ensembl (among a few others), but also returns a line of the protein sequence, which contains the amino acid sequence Glu-Asn-Ser-Gly (ENSG). In this simple example we could figure out the "right" result from this simple list, but in a more complicated example, this may not be the case. To narrow down the results of our search to those line which contain genuine Ensembl Gene IDs, we will define a pattern which generally describes these identifiers. Human Ensembl Gene IDs all start with ENSG, which is then followed by 11 numbers, so a regular expression which will match *any* human Ensembl Gene ID is: `'ENSG\d{11}'` - that is the exact letters `ENSG` followed by a sequence of exactly 11 digits (`\d`):

```
$ grep -P 'ENSG\d{11}' Q13315.txt
DR   Ensembl; ENST00000278616.9; ENSP00000278616.4;
DR   Ensembl; ENST00000452508.6; ENSP00000388058.2;
DR   Ensembl; ENST00000675843.1; ENSP00000501606.1;
DR   HPA; ENSG00000149311; Low tissue specificity.
DR   OpenTargets; ENSG00000149311; -.
DR   VEuPathDB; HostDB:ENSG00000149311; -.
DR   Bgee; ENSG00000149311; Expressed in corpus call
```

This more refined search removes the line of protein sequence information from the results, as well as the GeneTree database reference (which uses an identifier which looks a bit like the Ensembl Gene ID, but doesn't match it perfectly).

Now, if we want *just* the identifier, without any extraneous text, we can use the `grep` option `-o` which makes `grep` return only the text that matches the pattern:

```
$ grep -P -o 'ENSG\d{11}' Q13315.txt
ENSG00000149311
ENSG00000149311
ENSG00000149311
ENSG00000149311
ENSG00000149311
ENSG00000149311
ENSG00000149311
```

The result here repeats the Ensembl Gene ID several times, because it is found on several lines. If we just want to get it once, we can pipe these results into the `sort` command, which has an option (`-u`) to give only the unique entries in a list (the tool `uniq` does something similar, but requires the input to be sorted anyway, so we may as well just use a single tool):

```
$ grep -Po 'ENSG\d{11}' Q13315.txt | sort -u
ENSG00000149311
```

Exercise 2.3

Estimated time: 10 min

- Repeat the above procedure to retrieve the Ensembl Gene ID for the Rb protein
- Can you modify this procedure to retrieve the PubMed IDs associated with the RX lines we retrieved in exercise 2.2?
- (More difficult) How about the DOIs? What are the common features of DOIs which can be used to build a regular expression?

Other Text Processing Tools

As we've seen, much can be accomplished with `grep`, and the ability to not only search text but to effectively edit it and remove only the required sections make it a powerful tool, but Linux provides us with a well stocked armoury of such tools, allowing us to do so much more. We don't have space or scope to cover everything here, but included below are some selected highlights.

sort

The `sort` command takes a file and sorts its lines according to some natural order. By default this ordering is alphanumeric - it is possible to change this using options. With delimited text (e.g. tab- or comma-separated values) it is possible to choose the *field* you which to sort on (i.e. which column of the table). The field separator is defined using the `-t` option, and the column number to sort on is defined with the `-k` option.

uniq

The `uniq` command prints a file with repeated lines omitted. As mentioned above, in order for `uniq` to find repeated lines, the file must be sorted such that these lines are next to one another. The `-d` option will print *only* the repeated lines (i.e. the duplicates), and the `-n` option will include a count for each line (i.e. how many times it was found in the file).

cut

`cut` is used to select sections from each line of a file. This selection can be specified in terms of bytes, characters or fields, where fields can be thought of as the columns of a table. This makes `cut` particularly useful for selecting columns from a table.

The `-f` option is used to select particular fields (a comma-separated list can be used to retrieve multiple columns). By default, `cut` will expect fields to be separated by the tab character. This can be changed using the `-d` option (for delimiter).

Exercise 2.4

Estimated time: 10 min

- Download the example data table: https://github.com/sjcockell/mmb8052/raw/main/practicals/data/exar
- `sort` the file by the corrected p-value (the column name is `adj.P.Val`), and redirect the output to a file
    - To use the tab character as a field separator, you will need to add the `-t` option formatted like so: `-t $'\t'`
    - Are the results as you would expect?
    - Where do the column headings end up?
    - Can you figure out how to `sort` the column more correctly (read the `man` page - look for the "general numeric sort", which deals with scientific notation)
- Use a combination of `tail`, `sort`, `cut` and `head` to remove the column headers, find and return the names of the 10 genes with the largest positive log fold-change (`logFC`)

Scripting

In addition to interpreting commands directly at the command line, `bash` can also be used to execute a list of commands saved in a text file known as a *script*. A shell script is a file containing a list of commands which are executed one at a time, and is conventionally given the file extension `.sh`.

Scripting is useful for a number of reasons. For example it allows us to record and reuse our process because the commands we use are stored in a file. Furthermore, by introducing programming concepts such as variables, conditionals and iteration, we can automate our process, making us more efficient with our computational work.

We will use the `nano` text editor we first encountered in Practical 1 to make and edit our shell scripts.

Scripting Concepts

Variables

A *variable* is essentially a label referencing a location in memory holding an item of data. Once *declared*, variables can be used in operations, or passed to commands. The use of variables greatly increases the utility of scripts, and will often make them more readable and easier to edit.

We can create a variable in our script (or even at the command line) by giving it a name and assigning a value to it with the `=` operator:

```
MYVAR="Hello, world!"
```

The name of this variable is `MYVAR` and we have given it the value `"Hello, world!"`. It is important to note here that `bash` is quite picky about the formatting of this assignment - there can't be any spaces around the `=` (or `bash` will think you're trying to execute a command called `MYVAR`). Note that we've given our variable an ALL CAPS name - this isn't strictly necessary, but is the convention. One further point to make is that our variable name cannot contain characters other than alphanumerics and the underscore (`_`), or begin with a number. So `MYVAR`, `MY_VAR` and `MYVAR2` are all allowed variable names, but `MY VAR`, `MY-VAR` and `2MYVAR` are not.

Once we have created a variable, we can use it in subsequent lines of our script by referring to it. We do this with the `${}` operator:

```
echo ${MYVAR}
```

The curly braces here (`{}`) are not *absolutely* required, but can save introducing unexpected bugs in more complicated scripts. If we created a script with these two lines and ran it, we would see our script print "Hello, world!" to `STDOUT`.

Exercise 2.5

Estimated time: 3 min

- Create the 2 line script described above using `nano`. Save it as `my_first_script.sh`.
- Run the script as follows:

```
$ bash my_first_script.sh
```

- Do you get the expected output, as described above?
- If not, can you fix your script?

Environment

`bash` has a number of variables preset by default, this set of variables collectively is known as our *environment*. Some examples of standard `bash` environment variables are given in the table below.

Environment Variable	Description
HOME	Set to your home directory (~) at login
SHELL	Set to the path of your currently active shell
USER	Set to your Linux user name
PWD	Changes dynamically to be your present working directory

To find the current value of any of these variables, you can use `echo` in the same way we did above, for example:

```
$ echo ${SHELL}
/bin/bash
```

When we create a new variable, either at the command line or in our script, that variable enters this environment. In a script, the variable only lasts as long as the script is running, and then it gets "forgotten" by `bash` (this is known as the variable's *scope*). A variable created at the command line lasts until we exit our current command line session. While a variable is in your environment, there is no limit to the number of times you can reference it. It is also possible to change the value of an existing variable (note: be careful with variable *reuse*).

Special Variables

In a running script, we have access to dynamic run-time variables. These variables allow us to pass information into our script, for example consider the following script:

```
MY_FIRST_ARG=${1}
MY_SECOND_ARG=${2}
echo "The values you provided at the command line we
```

This script can be run as follows:

```
$ bash my_second_script.sh hello world
The values you provided at the command line were hel
```

The two variables referenced in the first two lines are special variables which store the command line arguments passed to the script. You can pass as many arguments as you like, but you'd have to write the code to handle them correctly. The `${#}` variable tells us how many arguments there are, and `${@}` provides a list of all of them.

Incorporating arguments into scripts can allow more general purpose scripts to be created, which can do the same job in many different ways in response to changing arguments. For example, we can create a script to download any UniProt entry:

```
UNIPROT=$1
echo "Downloading UniProt entry: ${UNIPROT}"
wget https://rest.uniprot.org/uniprotkb/${UNIPROT}.t
echo "Done downloading"
```

You can then invoke this script with:

```
$ bash my_third_script.sh P06400
Downloading UniProt entry: P06400
```

```
[...]
Done downloading
```

Exercise 2.6

Estimated time: 10 minutes

- Make the script above that takes a UniProt accession as an argument
- Modify it so you can also configure what type of data to download (e.g. `txt`, `fasta` or `gff`)
- Test that your script works as expected

Consider the following:

- What happens if you run the script without supplying arguments?
- How might you improve this behaviour?

The FOR Loop

The ability to *iterate*, or to perform a set of instructions repeatedly, is a key feature of programming languages. In `bash` we have the `for` flow control statement which enables iteration over a list of input variables. The list of commands to be executed at each iteration is enclosed between the keywords `do` and `done`. For example, a given set of UniProt entries could be downloaded with the following code:

```
for ACC in P34122 P19198 Q55618 P18778 Q9ZBZ1
do
```

```
  wget https://rest.uniprot.org/uniprotkb/${ACC}.txt
done
```

Here, the loop variable `ACC` is first assigned the value `P34122` and the code inside the `for` loop (between `do` and `done`) is executed - in this case our script simply runs `wget` with this `${ACC}` variable in the appropriate place in the UniProt URL. After this first iteration, the loop repeats but the `ACC` loop variable now takes the second value in the list (`P19198`). This process then repeats until the last item in the list of inputs is reached.

Once the `for` loop runs out of inputs (i.e. every entry in the list has been processed), the loop is terminated, and our script can move on to any lines after the `done`.

Exercise 2.7

Estimated time: 10 minutes

- Modify your script from exercise 2.6 to include a FOR loop so it can download multiple entries passed as arguments
- Test your script still works (use the list of UniProt entries above)
- Can you think of any further improvements you'd like to make to this script?
- Do you think it would also be possible to write a script to make using the UniProt search API (see practical 1) simpler?

Software Installation

So far the software we've used in Linux comes pre-installed with the system. Most Linux distributions will come with these utilities, and we can accomplish a number of complicated tasks with them, as we have already seen. It is often the case, however, that this pre-installed software is not sufficient for the task in hand. Fortunately there are a number of ways to install command-line software in Linux. We will be looking at a modern, cross-platform way of doing this in the next practical, but for today we will use the Ubuntu package manager, `apt`.

APT

APT stands for Advanced Package Tool, and the `apt` command-line tool is a utility for installing, updating, removing and otherwise managing software installation on an Ubuntu system - functioning like a command-line "App Store" (although everything is free!).

sudo

Installing software so that it can function correctly usually requires system-level changes. Therefore administrator privileges (the ability to modify any file on the file system) are required for `apt` to work properly. It is generally insecure to operate any computer as an admin user "full time", and Linux is no exception. This is why when you try to install software on Windows or MacOS you will be prompted for a password, even if you are logged in to an admin account. At the command line the `sudo` utility offers temporary admin uplift to the command that comes after it. The command to refresh our currently installed package list is `apt update` - this command does not work without admin privileges:

```
$ apt update
Reading package lists... Done
```

```
E: Could not open lock file /var/lib/apt/lists/lock
E: Unable to lock directory /var/lib/apt/lists/
W: Problem unlinking the file /var/cache/apt/pkgcach
W: Problem unlinking the file /var/cache/apt/srcpkgc
```

Notice the multiple 'permission denied' errors. Repeat using `sudo` this time, to provide the admin uplift. You will be prompted for your system password:

```
$ sudo apt update
[sudo] password for student:
Hit:1 http://gb.archive.ubuntu.com/ubuntu jammy InRe
[...]
Reading package lists... Done
```

Similarly, we can upgrade all the installed packages which require it. If you do this quickly enough after the command above, you will not need to enter your password again (by default you will not be prompted again until after 5 minutes of `sudo` inactivity).

```
$ sudo apt upgrade
```

Bioinformatics Software and APT

There are some popular bioinformatics software tools which are available to install via APT - if you want to know whether a tool can be installed via this route, you can try `apt search`:

```
$ apt search blast
[...]
ncbi-blast+/jammy 2.12.0+ds-3build1 amd64
  next generation suite of BLAST sequence search too
[...]
```

The above command returns lots of results, but among them is a command-line implementation of NCBI's BLAST+ package. The actual name of the package should be highlighted in the results (in green). We can install this package using `apt install` (this requires `sudo`):

```
$ sudo apt install ncbi-blast+
[...]
Do you want to continue? [Y/n]
```

APT will summarise the changes required to install the requested software (packages often have *dependencies* - other software or libraries they require to work properly - this is one reason why a package manager is so important, as this dependency resolution is automatically handled), and ask for confirmation that you wish to continue. Just hitting [ENTER] here is sufficient (the uppercase "Y" means that's the default option).

Once this installation is complete, we will have installed the BLAST suite of tools for command-line use:

```
# print out some help information for BLASTP
$ blastp -h
```

We will make use of BLAST in a future practical.

Putting it all together

For the final exercise in this practical, we will make use of the tools we've introduced above to download a set of sequences from UniProt and use some software we will install to build a multiple sequence alignment from the downloaded data. Although this sounds daunting at first, we can work through the process step-by-step.

Exercise 2.8

Estimated time: 15-20 mins

- Write a short script with a FOR loop (as above) to download this set of 12 protein sequences from UniProt:

P33568 D3ZS28 O55081 Q08999 P28749 P06400 Q64700 Q64701 P13405 Q24472 Q90600 G5EDT1

Note: download the `.fasta` sequence, not the `.txt` database entry. If you managed to modify the script in exercise 2.7 to take command line arguments, then it should be simple to modify this to download the correct data.

- Concatenate all of these sequence files together into a single multi-fasta file using `cat`:

```
$ cat *.fasta > all_seq.fasta
```

Note: This command will only work correctly if the FASTA files requested in the first step are the only `.fasta` files in your current working directory (otherwise sequences you don't want to include in your alignment will be incorporated in this file).

- Now install the multiple sequence alignment program called `muscle` - you can use `apt` for this (we are going to install the version called `muscle3`).

- Read the `muscle3` help file (`muscle3 --help`), and see if you can figure out how to get it to align the sequences in `all_seq.fasta` by following the "Basic usage".

- Use `less` to look at the difference between the input and the output.

Summary

This practical has introduced some advanced concepts which are useful when working at the Linux command line. The ability to write scripts, in particular, allows us to record our practice and therefore to reproduce it (and allow others to). We've also seen how to install software on our Linux system - providing new tools to experiment with, including some bioinformatics software. The final exercise here combined a lot of the lessons we've learned so far to achieve something quite complicated - this is a realistic 'bioinformatics' task.

Next week we will look at organising our work, and computational tools for managing projects and software.